ARCHITECTURE TECHNIQUE

AI-Viz Portal

Ecosystème Multi-Agents pour Dataviz IA

Conception d'une plateforme de visualisation

de benchmarks et métriques d'Intelligence Artificielle

Mars 2025

Table des matières

1. Résumé exécutif

2. Schéma du flux de données

3. Agent Data Engineer

- 3.1 Pipeline ETL
- 3.2 Schéma PostgreSQL
- 3.3 Stratégie de nettoyage

4. Agent Backend Supabase

- 4.1 Sécurité RLS
- 4.2 Edge Functions
- 4.3 Webhooks

5. Agent AI Domain Expert

- 5.1 KPIs définis
- 5.2 Taxonomie des modèles
- 5.3 Sources de benchmarks

6. Agent Frontend et UX

- 6.1 Stack technique
- 6.2 Bibliothèques de dataviz
- 6.3 Intégration Supabase

7. Agent Realtime et Performance

- 7.1 Architecture Realtime
- 7.2 Optimisations base de données
- 7.3 Pagination et virtualisation

8. Agent DevOps et Scalabilité

- 8.1 Stratégie de déploiement

8.2 Pipeline CI/CD

8.3 Sécurité

8.4 Plan de montée en charge

9. Roadmap de développement

9.1 Phase 1 : MVP

9.2 Phase 2 : Enrichissement

9.3 Phase 3 : Temps réel

9.4 Phase 4 : Plateforme complète

10. Conclusion

1. Résumé exécutif

Ce document présente l'architecture complète de **AI-Viz Portal**, une application de visualisation de données haute performance dédiée à l'écosystème de l'Intelligence Artificielle. Cette architecture a été conçue par un essaim de six agents spécialisés, chacun apportant son expertise dans son domaine technique respectif.

Stack technique global

Couche	Technologie	Rôle
Frontend	Next.js 14+ + Tailwind CSS	UI interactive, SSR/SSG
Backend	Supabase (PostgreSQL, Auth, Realtime)	BDD, Auth, Temps réel
Dataviz	Recharts + D3.js + Tremor	Graphiques interactifs
ETL	Python (pandas, PyMuPDF)	Extraction et transformation
Edge Functions	Deno/TypeScript	Calculs lourds, API tiers
Déploiement	Vercel + Supabase	Hosting serverless

L'objectif d'AI-Viz Portal est de fournir une plateforme centralisée pour l'analyse et la comparaison des performances des modèles d'IA. La plateforme ingère des données provenant de multiples sources (fichiers Excel, PDF techniques, APIs externes), les structure dans une base de données PostgreSQL optimisée, et les présente sous forme de visualisations interactives en temps réel.

2. Schéma du flux de données

L'architecture globale d'AI-Viz Portal suit un flux de données linéaire et modulaire, depuis l'ingestion des sources jusqu'à la présentation utilisateur finale.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        FLUX DE DONNEES AI-VIZ PORTAL                     │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  SOURCES              ETL                 SUPABASE           FRONTEND   │
│  ┌─────────┐        ┌─────────┐        ┌──────────┐       ┌──────────┐  │
│  │ Excel   │──────► │ Python  │──────► │PostgreSQL│──────►│Next.js   │  │
│  │ .xlsx   │        │pandas   │        │   + RLS  │       │+ React   │  │
│  └─────────┘        └─────────┘        └──────────┘       └──────────┘  │
│                                            │                  │          │
│  ┌─────────┐                               ▼                  ▼          │
│  │  PDF    │                          ┌──────────┐       ┌──────────┐  │
│  │ .pdf    │                          │  Edge    │       │ Recharts │  │
│  └─────────┘                          │Functions │       │  D3.js   │  │
│                                       └──────────┘       └──────────┘  │
│  ┌─────────┐                                                            │
│  │  APIs   │                                                            │
│  │HF/GitHub│                                                            │
│  └─────────┘                                                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Le flux de données se décompose en quatre étapes principales :

1. **Sources de données** : Fichiers Excel (.xlsx), documents PDF techniques, APIs externes (HuggingFace, GitHub, OpenAI, etc.)
2. **Pipeline ETL** : Extraction avec Python (pandas, PyMuPDF, OCR), transformation et nettoyage des données
3. **Stockage Supabase** : PostgreSQL avec RLS (Row Level Security), vues matérialisées, index optimisés
4. **Présentation Frontend** : Next.js avec Recharts/D3.js pour les visualisations interactives

3. Agent Data Engineer (Architecture et Ingestion)

3.1 Pipeline ETL complet

L'agent Data Engineer a conçu un pipeline ETL (Extract, Transform, Load) robuste capable d'ingérer des données depuis trois types de sources principales.

Extraction Excel

Les fichiers Excel sont traités avec la bibliothèque pandas et openpyxl. Le système attend des feuilles structurées selon des conventions prédéfinies :

Tableau 1 Configuration des feuilles Excel

Feuille	Colonnes requises	Colonnes numeriques

Benchmarks	model_name, dataset, accuracy, latency_ms	accuracy, latency_ms, throughput, f1_score
Costs	provider, model, input_cost_per_1k	input_cost_per_1k, output_cost_per_1k
Usage	endpoint, requests_count	requests_count, tokens_consumed

Extraction PDF (OCR)

Les documents PDF techniques sont traites via une chaine OCR combinant PyMuPDF pour le rendu et Tesseract pour la reconnaissance de caracteres. Des expressions regulieres specialisees extraient les metriques IA :

```
METRIC_PATTERNS = {
    'model_name': r'(?:Model|Modele)[\s:]*([^\n]+)',
    'accuracy': r'(?:Accuracy)[\s:]*(\d+\.?\d*)\s*%?',
    'latency': r'(?:Latency)[\s:]*(\d+\.?\d*)\s*(?:ms)?',
    'throughput': r'(?:Throughput)[\s:]*(\d+\.?\d*)\s*(?:tokens?/s)?'
}
```

Integration APIs tiers

Le connecteur API unifie implemente :

- **Rate limiting** : asyncio.Semaphore par fournisseur pour respecter les quotas
- **Retry logic** : Backoff exponentiel avec jitter pour la resilience
- **Providers supportes** : OpenAI, Anthropic, HuggingFace, Cohere, AI21

3.2 Schema PostgreSQL optimise

Le schema de base de donnees comprend dix tables principales, optimisees pour les requetes analytiques et le stockage de metriques temporelles.

Tableau 2 Tables principales du schema

Table	Description	Cles
providers	Fournisseurs IA (OpenAI, Anthropic, Meta...)	id (PK)
ai_models	Modeles IA disponibles	id (PK), provider_id (FK)
metric_types	Types de metriques standardisees	id (PK)
datasets	Datasets de benchmark	id (PK)
benchmarks	Resultats de benchmarks	id (PK), model_id (FK)
usage_stats	Statistiques d'utilisation temps reel	id (PK), user_id (FK)

data_sources	Sources de donnees ingerees	id (PK)
ingestion_logs	Logs detailles ETL	id (PK), source_id (FK)
model_comparisons	Comparaisons sauvegardees	id (PK)
data_versions	Versioning des donnees	id (PK)

3.3 Strategie de nettoyage et normalisation

Tableau 3 Regles de nettoyage des donnees

Probleme	Solution implementee
Valeurs manquantes	Remplissage par mediane, forward-fill, ou suppression selon le contexte
Unites inconsistants	Normalisation vers ms, USD, tokens/s, ratio (0-1)
Outliers	Detection MAD (Median Absolute Deviation) avec clipping
Doublons	Hash SHA-256 sur colonnes cles pour deduplication
Plages invalides	Clipping aux bornes acceptables par metrique

4. Agent Backend et Logic (Expert Supabase)

4.1 Architecture de securite et RLS

La securite repose sur le modele Row Level Security (RLS) de PostgreSQL, combine a l'authentification JWT de Supabase. Quatre roles sont definis avec des permissions granulaires.

Tableau 4 Modele de roles et permissions

Role	Description	Permissions
admin	Administrateur systeme	CRUD complet sur toutes les tables
analyst	Analyste de donnees	Lecture/Ecriture benchmarks, modeles
user	Utilisateur standard	Lecture benchmarks publics, creation visualisations
viewer	Visiteur/Invite	Lecture uniquement des donnees publiques

Exemple de politique RLS pour les benchmarks :

```
-- Benchmarks publics visibles par tous
CREATE POLICY "Public benchmarks are visible to all"
ON benchmarks FOR SELECT
TO authenticated
USING (visibility = 'public' AND status = 'published');

-- Les createurs voient leurs propres benchmarks
CREATE POLICY "Owners can view their benchmarks"
ON benchmarks FOR SELECT
TO authenticated
USING (owner_id = auth.uid());

-- Les admins voient tout
CREATE POLICY "Admins can view all benchmarks"
ON benchmarks FOR SELECT
TO authenticated
USING (EXISTS (
    SELECT 1 FROM users u
    WHERE u.id = auth.uid() AND u.role = 'admin'
));
```

4.2 Edge Functions

Les Edge Functions (Deno/TypeScript) executent la logique metier cote serveur avec acces direct a la base de donnees.

Fonction d'aggregation de benchmarks

Calcule les statistiques descriptives (moyenne, mediane, ecart-type, percentiles) pour un ensemble de resultats de benchmarks :

```
interface AggregateMetrics {
  mean: number;
  median: number;
  stdDev: number;
  min: number;
  max: number;
  count: number;
  p95: number;
  p99: number;
}
```

Fonction d'appel API HuggingFace

Proxy securise vers l'API HuggingFace avec rate limiting et gestion des erreurs :

```
const HF_API_URL = 'https://api-inference.huggingface.co/models/';

const hfResponse = await fetch(`${HF_API_URL}${model}`, {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${hfToken}` },
  body: JSON.stringify({ inputs, parameters }),
});
```

4.3 Database Webhooks

Les webhooks declenchent des actions lors des modifications de donnees. Une table webhook_events stocke les evenements pour traitement asynchrone avec mecanisme de retry.

```
CREATE TABLE webhook_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type TEXT NOT NULL,  -- INSERT, UPDATE, DELETE
    table_name TEXT NOT NULL,
    record_id UUID,
    old_record JSONB,
    new_record JSONB,
    processed BOOLEAN DEFAULT false,
    retry_count INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Trigger pour capturer les evenements
CREATE TRIGGER benchmark_webhook_trigger
    AFTER INSERT OR UPDATE OR DELETE ON benchmarks
    FOR EACH ROW EXECUTE FUNCTION capture_webhook_event();
```

5. Agent AI Domain Expert (KPIs et Taxonomie)

5.1 Indicateurs cles de performance (KPIs)

L'agent AI Domain Expert a defini 37 metriques reparties en six categories, couvrant l'ensemble des aspects d'evaluation des modeles d'IA.

Metriques de performance d'inference

Tableau 5 KPIs de performance d'inference

KPI	Definition	Unite	Importance
Tokens/seconde	Debit de generation de tokens	tokens/s	Critique
TTFT	Time To First Token	ms	Critique
TPOT	Time Per Output Token	ms/token	Elevée
Latence Totale	Temps de reponse complet	ms	Elevée

Benchmarks academiques

Tableau 6 Principaux benchmarks academiques

Benchmark	Description	Top Scores
MMLU	Massive Multitask Language Understanding (57 sujets)	GPT-4: 86.4%, Llama-3-70B: 82.0%
HumanEval	Generation de code Python (164 problemes)	GPT-4: 67%, CodeLlama-34B: 48.8%
GSM8K	Raisonnement mathematique (8.5K problemes)	GPT-4: 92%, Llama-3-70B: 93%
HellaSwag	Raisonnement commun	GPT-4: 95.3%
TruthfulQA	Veracite des reponses	GPT-4: 59%

Couts API (2024)

Tableau 7 Comparaison des couts API

Modele	Input ($/1M tokens)	Output ($/1M tokens)
GPT-4o	$5.00	$15.00
Claude 3 Sonnet	$3.00	$15.00
Llama 3 70B (Groq)	$0.59	$0.79
Mistral Large	$4.00	$12.00

5.2 Taxonomie des modeles IA

La taxonomie classe les modeles en cinq categories principales, avec des sous-categories et attributs descriptifs.

Tableau 8 Taxonomie des modeles IA

Categorie	Description	Exemples
LLM (Texte)	Large Language Models	GPT-4o, Llama-3, Claude, Mistral
Vision	Traitement et generation d'images	DALL-E 3, Stable Diffusion XL, SAM 2
Audio	Traitement et generation audio	Whisper, XTTS, MusicGen
Multimodal	Traitement multi-modalites	GPT-4V, Gemini, LLaVA
Embedding	Vecteurs de representation	text-embedding-3, BGE, CLIP

Top 5 modeles de reference

Tableau 9 Classement des modeles de reference

Rang	Modele	MMLU	HumanEval	Prix In/Out
1	GPT-4o	88.7%	90.2%	$5/$15
2	Claude 3.5 Sonnet	88.7%	92.0%	$3/$15
3	Llama-3.1-405B	88.6%	89.0%	Self-hosted
4	Llama-3.1-70B	86.0%	80.5%	Self-hosted
5	Mistral Large 2	86.5%	84.0%	$4/$12

5.3 Sources de benchmarks

Tableau 10 Sources de donnees recommandees

Source	Type	Frequence de mise a jour
HuggingFace Open LLM Leaderboard	Benchmark academique	Quotidienne
LMSYS Chatbot Arena	Evaluation ELO par utilisateurs	Quotidienne
MTEB Leaderboard	Benchmark embeddings	Hebdomadaire
GitHub API	Popularite (stars, forks)	Quotidienne

Artificial Analysis	Performance et prix	Quotidienne

6. Agent Frontend et UX (Interface Interactive)

6.1 Stack technique recommande

Tableau 11 Stack frontend

Technologie	Version	Justification
Next.js	14+ (App Router)	SSR/SSG natif, Server Components, streaming
React	18+	Concurrent features, Suspense, transitions
TypeScript	5+	Type safety, meilleure DX, maintenance
Tailwind CSS	3.4+	Utility-first, tree-shaking, dark mode natif
shadcn/ui	Latest	Composants accessibles, customisables

6.2 Bibliotheques de dataviz

Tableau 12 Choix des bibliotheques de visualisation

Cas d'usage	Bibliotheque	Priorite
Line/Bar/Area charts	Recharts	Critique
KPI Cards	Tremor	Elevée
Scatter/Heatmap complexes	D3.js	Moyenne
Radar/Composite charts	Recharts	Elevée

Recommandation : Utiliser Recharts pour 80% des graphiques standards, Tremor pour les KPI cards, et D3.js pour les visualisations specifiques complexes (heatmap, force graph).

Exemple Recharts

```
import { LineChart, Line, XAxis, YAxis, CartesianGrid,
         Tooltip, Legend, ResponsiveContainer } from 'recharts';

const ScoreEvolutionChart = ({ data }) => (
  <ResponsiveContainer width="100%" height={300}>
    <LineChart data={data}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="date" />
      <YAxis domain={[0, 100]} />
      <Tooltip />
      <Legend />
      <Line type="monotone" dataKey="score"
            stroke="hsl(var(--primary))" strokeWidth={2} />
    </LineChart>
  </ResponsiveContainer>
);
```

6.3 Integration SDK Supabase

L'integration utilise le pattern SSR/CSR hybride de Next.js 14 :

- **Server Components** : Fetch initial des donnees cote serveur
- **Client Components** : Interactivite, filtres, mises a jour temps reel

```
// Configuration du client Supabase
import { createBrowserClient } from '@supabase/ssr';

export const createClient = () => {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
};

// Hook pour data fetching avec TanStack Query
function useBenchmarks(filters) {
  return useQuery({
    queryKey: ['benchmarks', filters],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('benchmarks')
        .select('*, models(*)')
        .match(filters);
      if (error) throw error;
      return data;
    },
  });
}
```

Architecture de navigation

```
AI-Viz Portal
├── /dashboard           → Dashboard principal
│   ├── Overview         → Vue d'ensemble
│   ├── Comparison       → Comparaison modeles
│   └── Trends           → Tendances temporelles
├── /modeles             → Liste des modeles
│   ├── /[id]            → Detail modele
│   └── /compare         → Comparaison cote-a-cote
├── /benchmarks          → Historique benchmarks
├── /leaderboard         → Classement global
└── /settings            → Preferences utilisateur
```

7. Agent Realtime et Performance

7.1 Architecture Realtime Supabase

Supabase Realtime utilise le mecanisme PostgreSQL LISTEN/NOTIFY combine avec des WebSockets pour diffuser les changements de donnees en temps reel.

Flux de donnees : PostgreSQL → WAL (Write-Ahead Log) → Realtime Server → WebSocket → Client

Configuration des channels

```
export const CHANNEL_CONFIG = {
  benchmarks: {
    name: 'benchmark-results-channel',
    table: 'benchmark_results',
    events: ['INSERT', 'UPDATE'],
    filter: 'status=eq.completed',
  },
  models: {
    name: 'model-versions-channel',
    table: 'model_versions',
    events: ['INSERT', 'UPDATE', 'DELETE'],
  },
  metrics: {
    name: 'performance-metrics-channel',
    table: 'performance_metrics',
    events: ['INSERT'],
    filter: 'created_at=gte.' + new Date(Date.now() - 3600000).toISOString(),
  },
};
```

Hook Realtime avec batching

Le batching (regroupement) des evenements sur 100ms evite les re-renders excessifs :

```
export function useRealtimeBenchmarks(options = {}) {
  const { batchInterval = 100 } = options;
  const batchQueueRef = useRef([]);

  const processBatch = useCallback(() => {
    if (batchQueueRef.current.length === 0) return;
    const batch = [...batchQueueRef.current];
    batchQueueRef.current = [];
    batch.forEach(payload => {
      if (payload.eventType === 'INSERT') onInsert?.(payload);
      if (payload.eventType === 'UPDATE') onUpdate?.(payload);
    });
  }, [onInsert, onUpdate]);

  useEffect(() => {
    const channel = supabase
      .channel(CHANNEL_CONFIG.benchmarks.name)
      .on('postgres_changes', { event: '*', schema: 'public', table: 'benchmarks' },
        (payload) => queueEvent(payload)
      )
      .subscribe();
    return () => channel.unsubscribe();
  }, []);
}
```

7.2 Optimisations base de donnees

Indexation recommandee

```
-- Index pour les requetes temps reel (filtrage par date)
CREATE INDEX CONCURRENTLY idx_benchmark_results_created_at
    ON benchmark_results(created_at DESC);

-- Index composite pour les filtres courants
CREATE INDEX CONCURRENTLY idx_benchmark_results_model_dataset
    ON benchmark_results(model_id, dataset_id, created_at DESC);

-- Index GIN pour les requetes JSONB
CREATE INDEX CONCURRENTLY idx_benchmark_results_metadata_gin
    ON benchmark_results USING GIN(metadata);

-- Index BRIN pour les grandes tables temporelles
CREATE INDEX CONCURRENTLY idx_metrics_brin
    ON performance_metrics USING BRIN(recorded_at);
```

Vues materialisees

```
-- Vue: Statistiques par modele (rafraichissement toutes les 5 min)
CREATE MATERIALIZED VIEW mv_model_stats AS
SELECT
    model_id,
    COUNT(*) as total_benchmarks,
    AVG(accuracy) as avg_accuracy,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY accuracy) as median_accuracy,
    AVG(latency_ms) as avg_latency,
    AVG(throughput) as avg_throughput
FROM benchmark_results
WHERE status = 'completed'
GROUP BY model_id;

-- Rafraichissement automatique via cron
SELECT cron.schedule('refresh-model-stats', '*/5 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY mv_model_stats');
```

7.3 Pagination et virtualisation

Pagination cursor-based

La pagination cursor-based offre une complexite O(1) contre O(n) pour l'offset traditionnel :

Tableau 13 Comparaison des strategies de pagination

Critere	Offset	Cursor-Based
Performance	O(n) - ralentit avec la page	O(1) - constante
Consistance	Risque de doublons/sauts	Stable en temps reel
Navigation	Page aleatoire possible	Sequentielle uniquement

Performance attendue

Tableau 14 Gains de performance apres optimisation

Scenario	Avant	Apres	Gain
Chargement initial	3-5s	1-2s	60%
Mise a jour temps reel	200-500ms	50-100ms	75%

Rendu 1000 points	100-200ms	16-33ms	85%
Requete aggregation	2-5s	100-500ms	90%

8. Agent DevOps et Scalabilite

8.1 Strategie de deploiement

La plateforme utilise trois environnements distincts avec une promotion automatique via CI/CD.

Tableau 15 Architecture des environnements

Environnement	Plateforme	Usage
Development	Local + Docker	Developpement et tests
Staging	Vercel + Supabase Staging	Tests d'integration
Production	Vercel + Supabase Production	Deploiement final

Pourquoi Vercel pour Next.js ?

- Optimisation Next.js native
- Edge Functions integres
- Support natif des Server Components
- Cold start inferieur a 50ms

8.2 Pipeline CI/CD

La pipeline GitHub Actions comprend 8 jobs parallelises :

1. Lint et Type Check (~3 min)
2. Unit Tests avec Jest (~5 min)
3. Integration Tests (~8 min)
4. Security Scan avec Trivy (~5 min)
5. E2E Tests avec Playwright (~15 min)
6. Deploy Staging (sur branche develop)
7. Deploy Production (sur branche main)

Dockerfile multi-stage

```
# Multi-stage build: 150MB final vs 2GB
FROM node:20-alpine AS base
FROM base AS deps
RUN npm ci

FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
RUN npm run build

FROM base AS runner
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

8.3 Securite

Tableau 16 Mesures de securite

Attaque	Protection	Implementation
DDoS	Rate Limiting	Vercel Edge + Upstash Redis
SQL Injection	Parametrisation	Supabase client
XSS	CSP + Sanitization	next.config.js + DOMPurify
CSRF	Token validation	Double-submit cookie
Data Leak	RLS Policies	Supabase Row Level Security

Headers de securite

```
// next.config.js
async headers() {
  return [{
    source: '/:path*',
    headers: [
      { key: 'X-Frame-Options', value: 'DENY' },
      { key: 'X-Content-Type-Options', value: 'nosniff' },
      { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
      { key: 'Content-Security-Policy', value: "default-src 'self'..." },
    ],
  }];
}
```

8.4 Plan de montee en charge

Tableau 17 Evolution des couts par phase

Phase	Utilisateurs	Cout mensuel	Configuration
1	0-1K	0€	Vercel Hobby + Supabase Free
2	1K-10K	45€	Vercel Pro + Supabase Pro
3	10K-100K	600€	Read replicas, CDN externe
4	100K+	1000€+	Enterprise, multi-region

Backup et recovery

Tableau 18 Strategie de backup

Type	Frequence	Retention	Stockage
Database Full	Quotidien (2h)	30 jours	S3 Standard-IA
Database Incremental	Toutes les 4h	7 jours	S3 Standard
PITR	Continu	7 jours	Supabase interne

RPO (Recovery Point Objective) : moins de 4h de donnees perdues maximum **RTO** (Recovery Time Objective) : moins de 4h pour reprise complete

9. Roadmap de developpement

9.1 Phase 1 : MVP - Fondation (Semaines 1-4)

Objectif : Plateforme fonctionnelle avec ingestion et affichage simple des benchmarks.

Livrables

- Pipeline ETL basique (Excel/CSV)
- Schema PostgreSQL minimal
- Authentification Supabase
- Dashboard simple avec tableaux
- Visualisations basiques (line/bar charts)

Stack minimal

Frontend: Next.js + Tailwind + Recharts | Backend: Supabase (Auth + PostgreSQL) | ETL: Python pandas

Metriques supportees

MMLU, HumanEval (import manuel) | 50 modeles de reference

9.2 Phase 2 : Enrichissement (Semaines 5-8)

Objectif : Ajout des sources automatiques et amelioration de l'UX.

Livrables

- Integration APIs (HuggingFace, GitHub)
- Extraction PDF/OCR
- Filtres dynamiques avances
- Mode sombre/clair
- Tableaux de comparaison cote-a-cote
- 150+ modeles

Nouvelles fonctionnalites

Realtime subscriptions (basique) | Edge Functions (aggregations simples) | RLS policies completes | Export CSV/JSON

Metriques additionnelles

GitHub Stars, HF Downloads | Couts API (prix par 1K tokens) | Latence, Throughput

9.3 Phase 3 : Analyse et temps reel (Semaines 9-12)

Objectif : Dashboard analytique complet avec mises a jour temps reel.

Livrables

- Realtime subscriptions avancees
- Vues materialisees pour performances
- Scores composites (Intelligence, Speed, Value)
- Leaderboard interactif
- Alertes et notifications
- 300+ modeles

Optimisations

Indexation avancee | Pagination cursor-based | Virtualisation des listes | Web Workers pour calculs

Nouveaux composants

Heatmaps de comparaison | Graphiques radar | Tendances temporelles | Predictions simples

9.4 Phase 4 : Plateforme complete (Semaines 13-16+)

Objectif : Plateforme temps reel complete avec scalabilite entreprise.

Livrables

- CI/CD complet
- Monitoring et alerting
- Multi-tenant (equipes)
- API publique
- Documentation complete
- 500+ modeles

Architecture finale

```
┌──────────────────────────────────────────────────────────────┐
│                    AI-VIZ PORTAL v1.0                        │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │   Web App    │  │   API        │  │   Docs       │       │
│  │   (Next.js)  │  │   (REST)     │  │   (Swagger)  │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Realtime Dashboard + Alerts + Export + Sharing      │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  ETL Pipeline + Edge Functions + Webhooks            │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Supabase (Auth + DB + Realtime + Storage)           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

10. Conclusion

Cette architecture Swarm pour **AI-Viz Portal** fournit une fondation solide et scalable pour une application de dataviz IA moderne. Les six agents specialises ont couvert l'ensemble des aspects techniques necessaires au developpement d'une plateforme professionnelle.

Recapitulatif des contributions

Tableau 19 Contributions par agent

Agent	Expertise	Livrables cles
Data Engineer	Pipeline ETL et modelisation	Pipeline robuste, schema PostgreSQL optimise
Backend Expert	Securite et logique serveur	RLS, Edge Functions, Webhooks
AI Expert	KPIs et taxonomie	37+ KPIs, taxonomie complete des modeles
Frontend Expert	Interface utilisateur	Stack moderne (Next.js + Tailwind + Recharts)
Realtime Expert	Performance et temps reel	Subscriptions temps reel, optimisations
DevOps Expert	Deploiement et scalabilite	CI/CD, securite, backup/recovery

La roadmap en quatre phases permet un developpement iteratif, du MVP a la plateforme complete, avec des jalons clairs et mesurables. Chaque phase ajoute des fonctionnalites significatives tout en maintenant la stabilite et la

performance du systeme.

Points forts de l'architecture

- **Modularite** : Separation claire des responsabilites entre les couches
- **Scalabilite** : Architecture serverless avec scaling automatique
- **Securite** : RLS, authentification JWT, headers de securite
- **Performance** : Indexation, vues materialisees, pagination optimisee
- **Temps reel** : WebSockets pour mises a jour instantanees